DEAD Z THROTTLE

Complete Development Documentation — v3.0

Associate Unity Developer Technical Test

Unity 6.3 LTS • Android • URP • Portrait • GLES3 / Vulkan

Dhiyanesh G — IIT Guwahati

1. Project Overview

Dead Z Throttle is a top-down portrait mobile car game built for Android. The player drives a car using swipe/drag controls and attempts to hit as many zombies as possible within a configurable countdown timer. Each hit increments the score. When the timer reaches zero a final score screen is shown with a restart button.

Core Requirements Checklist

Item	Status	Notes
Car Controller Integration	**Done**	Prometeo adapted to Hot Slide swipe style
Top-Down Angled Camera	**Done**	Isometric-style, follows car smoothly
Swipe / Drag Input	**Done**	Camera-relative world-space direction
Countdown Timer (configurable)	**Done**	Inspector field on GameManager
Score Counter (HUD)	**Done**	Updates in real time via UnityEvent
Zombie Wandering (min 10)	**Done**	Object pool — 10 active, 5 reserve
Ragdoll on Hit	**Done**	Unity Ragdoll Wizard, no paid middleware
Zombies Stay in Scene	**Done**	Ragdoll remains — pool spawns replacement
Game Over + Restart	**Done**	Scene reload, clean state reset
FPS Counter Visible	**Done**	Top-right corner, 0.5s average
Android APK	**Done**	Tested on Infinix X6525B, Android 13
Target ≥58 FPS	**Done**	Stable 60 FPS — Application.targetFrameRate = 60

2. Project Setup

Unity & Build Configuration

- Unity Version: 6.3 LTS
- Render Pipeline: Universal Render Pipeline (URP) — Mobile_RPAsset assigned
- Platform: Android, Portrait locked
- Minimum API Level: Android 8.0 (API 26)
- Target API Level: Automatic (highest installed)
- Scripting Backend: IL2CPP, Target Architecture: ARM64 + ARMv7
- Graphics API: Auto (Vulkan + OpenGLES3)
- Active Input Handling: Both (editor testing requires old system; device uses new)
- Texture Compression: ASTC

Assets Used

- **Car:** Prometeo Car Controller

- **Zombies:** Supercyan Character Pack Zombie Sample v3.0

Folder Structure

```
Assets/_Game/Scripts/
Assets/_Game/Scripts/Editor/   ← ShaderVariantStripper lives here
Assets/_Game/Prefabs/
Assets/_Game/Scenes/
Assets/_Game/Materials/
```

Arena

- Floor: Plane scaled (4,1,4) — 40×40 unit play area
- 4 invisible wall cubes at arena edges tagged "Wall" — keep zombies inside
- No NavMesh — flat open arena with no obstacles; simple wandering is sufficient

3. Car Controller Adaptation (Prometeo → Hot Slide Style)

The Prometeo Car Controller was adapted without replacing its physics internals. A Bridge pattern separates concerns: SwipeInputHandler reads touch input, CarInputBridge translates it into Prometeo method calls. Prometeo's built-in keyboard input was neutralised to prevent it fighting our input.

Changes Made to PrometeoCarController.cs

- decelerateingCar → [HideInInspector] public — required by CarInputBridge
- GoForward() and GoReverse() → made public — called by CarInputBridge
- ResetSteeringAngle() rewritten — removed Input.GetAxis("Horizontal") dependency
- Keyboard else-block stripped of ThrottleOff / InvokeRepeating auto-calls — these were overriding our GoForward() every frame and preventing movement
- useTouchControls = false set in CarInputBridge.Awake()

Input Model — Hold & Drag (Hot Slide Style)

Player touches anywhere on screen and drags. The drag direction in screen space is converted to a world-space direction using the camera's forward and right axes. This means dragging "up" always means "toward where the camera is looking" regardless of camera angle — making controls intuitive at any view angle.

SwipeInputHandler.cs

Single Responsibility: reads touch/mouse drag, outputs joystickDirection (Vector3) and IsTouching (bool). Converts screen delta to world direction via GetWorldDirection() using Camera.main.transform.forward and .right flattened to the ground plane. Dead zone of 10px prevents accidental micro-movements from steering. UNITY_EDITOR block provides mouse drag fallback for editor testing — stripped from builds.

CarInputBridge.cs

Single Responsibility: translates SwipeInputHandler output into Prometeo calls. While IsTouching: cancels deceleration, calls GoForward(). Steering: Vector3.SignedAngle between car.transform.forward and joystickDirection — positive = TurnRight, negative = TurnLeft, within 5° = ResetSteeringAngle. DecelerateCarProxy() wrapper needed because InvokeRepeating cannot directly call methods on other MonoBehaviours.

SOLID Principles Applied

- Single Responsibility — SwipeInputHandler reads input only; CarInputBridge translates only
- Open/Closed — Prometeo extended via bridge, internal physics untouched
- Dependency Inversion — CarInputBridge depends on SwipeInputHandler abstraction
- Interface Segregation — IHittable is a single focused method

4. Zombie System

Design Decisions

NavMesh was deliberately not used. The arena is flat and open with no obstacles — simple random direction wandering achieves the same visual result with far less complexity and zero baking setup. This keeps the system understandable and lightweight.

ZombieWander.cs — Random Wandering

Picks a random direction using Sin/Cos on a random 0-360° angle (uniform distribution across all directions — using Random.Range(-1,1) for X and Z separately would bias toward diagonals). Walks that direction for a random 2-4 seconds, then picks a new one. OnEnable() resets state on each pool activation — Awake() would not work here because it only runs once, but OnEnable() runs every time SetActive(true) is called. On wall collision, picks a new direction immediately. OnDeath() stops movement and disables animator.

ZombieRagdoll.cs — Ragdoll Transition

Implements IHittable. On OnHit(): isDead guard prevents double-hit (OnCollisionEnter can fire multiple times per frame at high speed). Calls ZombieWander.OnDeath(), activates ragdoll physics, applies impulse force to all bone Rigidbodies. SetRagdollActive() skips the root Rigidbody (rb.transform == transform) — that is the capsule collider for the whole zombie, not a ragdoll bone. Bone colliders are disabled during animation to prevent inter-bone collisions. The ragdolled zombie stays in scene permanently — it never returns to the pool.

ZombiePool.cs — Object Pooling

Pre-creates (activeCount + 5) = 15 zombies at Start(), all SetActive(false). Activates first 10 immediately. When a zombie is hit, OnZombieDied() waits respawnDelay seconds then activates one reserve zombie at a random arena position. Pool is one-directional: inactive → active only. Hit zombies stay in scene as ragdolls. GetInactiveZombie() uses activeInHierarchy (not activeSelf) to correctly detect truly inactive objects regardless of parent state. Returns null gracefully if pool is exhausted — only happens near end of round.

Pool Flow Summary

```
Game starts → 15 zombies created, 10 activated
Zombie hit → ragdolls, stays on ground
             → ZombiePool.OnZombieDied() called
             → waits 2.5 seconds
             → finds inactive zombie from pool
             → SetActive(true) → OnEnable() resets it → wanders again
```

5. Camera System

CarCameraFollow.cs uses LateUpdate (not Update) to follow the car — LateUpdate runs after all Update() calls, ensuring the car has moved before the camera repositions. Using Update() would cause one-frame jitter. Vector3.Lerp with smoothSpeed * Time.deltaTime gives frame-rate independent smooth follow. Camera is angled in isometric style matching Hot Slide's view rather than pure 90° top-down.

- Offset set in Inspector to match desired angle — e.g. (-8, 13, -8) for Hot Slide style
- Smooth Speed: 5 — responsive without snapping

6. Game Manager & HUD

GameManager.cs

Singleton. Owns all game state: IsPlaying, Score, TimeLeft. Uses UnityEvents (onScoreChanged, onTimerChanged, onGameOver) so HUD subscribes without GameManager knowing about it — fully decoupled. Application.targetFrameRate = 60 set in Awake() to prevent Android's default 30 FPS

battery-saving mode. AddScore() guards with if (!IsPlaying) to prevent post-game scoring on collision edge cases.

HUDManager.cs

Purely presentational — subscribes to GameManager events in Start(), never changes game state. Timer shows ceiling int (CeilToInt) so it displays 60 not 59.9. Turns red under 10 seconds. FPS averaged over 0.5 second windows — per-frame measurement would flicker too fast to read. Restart calls SceneManager.LoadScene(buildIndex) which fully reloads all objects cleanly.

7. Android Build & Performance

Shader Compilation Fix — ShaderVariantStripper.cs

Unity 6 includes DOTS_INSTANCING_ON variants for every URP shader by default. DOTS instancing is designed for games with thousands of entities — completely unnecessary here. These variants are large enough to cause the GLES3 shader compiler to run out of memory during the build. ShaderVariantStripper.cs (in Assets/_Game/Scripts/Editor/) implements IPreprocessShaders and strips all DOTS_INSTANCING_ON variants before compilation starts. Loop runs backwards when removing items to avoid index shifting errors.

Mobile_RPAsset Settings for Performance

- HDR: Off
- Anti-Aliasing: Disabled
- Render Scale: 0.8
- Shadow Distance: 20, Cascade Count: 1
- Additional Lights Cast Shadows: Off
- Soft Shadows: Off
- Reflection Probe Blending: Off
- Probe Atlas Blending: Off
- Box Projection: Off
- Post-processing: Bloom and Motion Blur disabled

Result

- Device tested: Infinix Smart 8 Pro (X6525B), Android 13, OpenGLES3
- FPS: Stable 60 — target of ≥58 met
- APK installs and runs cleanly

8. Scripts Summary

Script	Location	Responsibility
`IHittable.cs`	Interfaces/	Contract for car-hittable objects
`CarCameraFollow.cs`	Scripts/	Smooth isometric camera follow
`SwipeInputHandler.cs`	Scripts/	Touch drag → world-space direction
`CarInputBridge.cs`	Scripts/	Swipe direction → Prometeo calls
`CarCollisionHandler.cs`	Scripts/	Collision detection + IHittable caller
`ZombieWander.cs`	Scripts/	Random direction wandering
`ZombieRagdoll.cs`	Scripts/	Ragdoll toggle + IHittable impl
`ZombiePool.cs`	Scripts/	Object pool, 10 active zombies
`GameManager.cs`	Scripts/	Timer, score, game state, events
`HUDManager.cs`	Scripts/	UI display, FPS counter, restart
`ShaderVariantStripper.cs`	Scripts/Editor/	Strips DOTS_INSTANCING_ON at build

9. Issues Encountered & Resolutions

Issue / Decision	Resolution
URP grey screen on start	Assigned Mobile_RPAsset in Quality Settings for Android
CameraFollow name conflict with Prometeo	Renamed our script to CarCameraFollow.cs
Prometeo keyboard block fighting our input	Stripped ThrottleOff/InvokeRepeating auto-calls from keyboard else-block in Prometeo Update()
Car not moving at all	useTouchControls = false disabled Prometeo UI but keyboard else-block still called ThrottleOff() every frame
Car spinning in circles	GoForward() called on pure horizontal drag — added 5° threshold in HandleSteering()
Shader OOM error on GLES3 build	Added ShaderVariantStripper.cs in Editor folder to strip DOTS_INSTANCING_ON variants at build time
APK not installing (Android 13)	Changed Active Input Handling to New Input System — resolved signing/install issue
FPS 30 on device	Added Application.targetFrameRate = 60 in GameManager.Awake()
Score incrementing multiple times per hit	isDead guard in ZombieRagdoll.OnHit() prevents double-hit; AddScore moved to ZombieRagdoll
Ragdoll kinematic warning on reset	Clear velocity only when !rb.isKinematic before setting kinematic state
ZombieAI over-engineered with NavMesh	Replaced with simple random direction wandering — NavMesh unnecessary for flat open arena
"Object pooling" was just respawning same zombie	Rebuilt as genuine pool: pre-created list, hit zombies stay as ragdolls, reserve activates as replacement

10. Known Limitations

- Pool has 5 reserve zombies — if player hits more than 5 zombies near end of round, no replacement spawns (graceful, not a crash)

- Zombie wander does not check arena bounds — relies on wall collisions to redirect. A zombie could theoretically walk into a corner and oscillate briefly
- No reverse on car — matches Hot Slide reference intentionally
- UI bottom area shows grey — planned for visual polish in future iteration